第 6 章練習詳解：CPU 排程

OS06-E01　非搶占排程計算

FCFS 依題設順序執行：P1 [0,8)、P2 [8,12)、P3 [12,14)。SJF 依 burst 由短到長執行：P3 [0,2)、P2 [2,6)、P1 [6,14)。因全部在 0 抵達且沒有 I/O，周轉時間＝完成時間，等待時間＝完成時間−CPU 需求。

工作	FCFS 完成／等待／周轉	SJF 完成／等待／周轉
P1	8／0／8	14／6／14
P2	12／8／12	6／2／6
P3	14／12／14	2／0／2

FCFS 平均等待 `(0+8+12)/3=20/3`；SJF 為 `(6+2+0)/3=8/3`，少 4。平均周轉分別為 34/3、22/3；總完成時間均為 14。

同時抵達、單 CPU、每個工作恰有一次已知實際長度的 burst、沒有 I/O 與切換成本，是此交換論證的前提。若只估計 burst 而把順序排錯，就不能保證實際平均等待仍最短。

OS06-E02　Round Robin

q＝2 時依初始 queue 選取，尚未完成的工作回到隊尾：

`P1 [0,2) → P2 [2,4) → P3 [4,6) → P1 [6,8) → P3 [8,9) → P1 [9,10)`。

工作	完成	等待＝完成−需求	首次 CPU 回應
P1	10	5	0
P2	4	2	2
P3	9	6	4

平均等待為 13/3。q＝1 時，每格各 1 單位：

區間起點	0	1	2	3	4	5	6	7	8	9
執行工作	P1	P2	P3	P1	P2	P3	P1	P3	P1	P1

工作	完成	等待	首次 CPU 回應
P1	10	5	0
P2	5	3	1
P3	8	5	2

平均等待仍是 13/3，但個別工作及首次回應不同。排除最後的結束點後，q＝2 有 5 個相鄰執行區間的邊界；q＝1 有 9 個時間片邊界，其中末兩片都是 P1，故只發生 8 次切換至不同工作。兩個時間線均無 idle，需求總和為 10。

OS06-E03　優先權與 starvation

1. 令 L 於 0 就緒，每個整數 k 都有高優先權 H_k 抵達並需要 1 ms；高者可搶占。若依序讓 H_k 占用 [k,k+1)，L 永遠無法得到 CPU。這是一個無限執行反例；單次觀測很長的等待不能證明無限飢餓。
2. L 最初優先權為 0，每等 2 ms 加 1；持續等待時於 t＝20 達最高級 10。若最高級按 FIFO 且新到者不能插隊，L 不再持續被新來者越過。
3. 假設 t＝20 時正在執行的工作尚需至多 B、另有 q 個同級 ready 工作排在 L 前面，每個已選中的工作占用至多 B。故從 0 起，到 L 首次執行至多等待 `20+(q+1)B` ms。此式的 q 不包含正在執行者。若要在事前給共同數值上界，還須限制 q≤N，得到 `20+(N+1)B`。只寫「有 aging」而沒有有限前方工作數、服務上界與同級公平規則，不足以推出這個界。

OS06-E04　HRTICK CFS 逐事件推演

1. 可執行實體數 n＝2，不超過 `6/0.75=8`，故 period＝6 ms。兩人權重相同，ideal runtime 為 `6×1024/(1024+1024)=3` ms。nice＝0 時權重為 1024，執行 1 ms 對 vruntime 的增量亦為 1 ms。
2. 下表是在該時刻處理完事件並選出下一個 current 後的狀態；ready 欄不包含 current。`min_vruntime` 不倒退。

t	current	其他 ready（工作@vruntime）	min_vruntime	選取／切換原因
0	P1	P2@0	0	同值，按入樹次序選 P1
3	P2	P1@3	0	P1 的 HRTICK 片段結束
6	P1	P2@3	3	P2 的片段結束；同值選先入樹的 P1
9	P3	P2@3、P1@6	3	P3 入列後 `hrtick_update` 重算片額並要求重新排程
11	P2	P1@6	3	P3 完成；選較小 vruntime
12	P1	無	6	P2 完成
14	無	無	8	P1 完成

3. 時間線為 `P1 [0,3) → P2 [3,6) → P1 [6,9) → P3 [9,11) → P2 [11,12) → P1 [12,14)`；完成時間依 P1、P2、P3 為 14、12、11。t＝8 沒有題設中會導致切換的完成、喚醒或 HRTICK 事件；若普通 tick 在該時刻發生，也不能只憑 tick 存在就推論必然重排。
4. t＝9 的 placement 為 `max(舊 vruntime, min_vruntime−3)=max(0,3−3)=0`。P3 入列使 runnable 數變成 3，各人的 ideal runtime 因此從 3 ms 降為 2 ms；P1 這次執行已用 3 ms。Linux 3.2 的 `enqueue_task_fair` 呼叫 `hrtick_update`，發現新片額減已用時間為負，便先對 current 設定重新排程。依題設順序，後續 `check_preempt_wakeup` 看見已要求重新排程而返回；選取時 P3 的 vruntime 最小，故選 P3。若先處理原 timer 到期，也可能先要求重新排程；不能把此例原因寫成喚醒比較。P3 在 t＝9 喚醒前不能執行。

OS06-E05　週期 tick 與嚴格大於

1. 第一個 current P1 的 `delta_exec` 在 t＝3 為 3 ms，在 t＝4 為 4 ms。Linux 3.2 的 `check_preempt_tick` 第一條件為 `delta_exec > ideal_runtime`；因 ideal runtime＝3 ms，前者 3>3 不成立，後者 4>3 成立。
2. P1 [0,4)；P2 [4,8) 並完成；P1 [8,9)。P3 在 t＝9 醒來後 placement 得 vruntime＝2，當時 P1 為 5，差值 3>1，P3 搶占並在 [9,11) 完成；P1 [11,14) 完成。完成時間為 P1＝14、P2＝8、P3＝11。
3. Ideal runtime 是模型中進行排程判斷所用的時間尺度；一般 tick 僅在檢查點判斷，且嚴格大於不在相等時成立。HRTICK 可以安排相應時間點的檢查，但完成、喚醒及其他可搶占事件也可能改變時間線，不能將 ideal runtime 當作無條件的精確切換保證。
4. 這小題改回 OS06-E04 的 HRTICK 設定，並令 P3 於 8.5 醒來。P3 入列後 runnable 數增至 3，新片額為 2 ms；P1 在此輪已執行 2.5 ms，`hrtick_update` 因此要求重新排程。原來的 P1 [6,9) 先變為 [6,8.5)；接著 `P3 [8.5,10.5) → P2 [10.5,11.5) → P1 [11.5,14)`。P3、P2、P1 分別於 10.5、11.5、14 完成。

OS06-E06　CPU 公平與同步語意

1. 總 weight＝1024＋335＝1359。A 的 ideal runtime 為 $6\times1024/1359\approx4.521$ ms，B 為 $6\times335/1359\approx1.479$ ms；CPU 份額分別約為 75.35%、24.65%。這是所給權重與 period 的計算，不是兩人固定相同的 quantum。
2. 不能。抽象的 load／add／store 中，A、B 可都讀到 counter＝0，各自加一後都寫回 1，遺失一次更新。排程權重只處理 CPU 分配，不使複合更新原子化。在 C/C++ 中，未同步的衝突非原子存取可構成 data race，行為未定義；該抽象交錯不是對實際輸出的保證。
3. B 完成 buffer 初始化後以 atomic release store 設 ready，A 用讀到該次發布或其 release sequence 的 atomic acquire load 觀察 ready；在對應語言與 API 的 happens-before 契約下，B 之前的寫入對 A 可見。buffer 仍須存活，且 B 不可在 A 讀取期間無同步地再次修改；也可用具有相應排序與物件生命週期保證的 mutex／condition-variable 協定。
4. 所有相關讀寫遵循同一把 mutex 時，同一時間至多一個持有者執行受保護區段。若 API 的 unlock／lock 另外提供 release／acquire 同步，受保護資料也可透過對應操作發布；仍須確認發布與讀取位於相同保護協定中，並正確處理失敗、等待和釋放。
5. Memory barrier 限制指定操作的順序，但本身不保證臨界區互斥；atomic operation 讓其指定位置的操作符合所選原子與記憶體順序契約，單一 atomic 不自動保護多位置不變量。Mutex 可提供所有權與區段互斥，其可見性依 API 定義；semaphore 控制 permits，binary semaphore 在正確配對協定下可序列化工作，但通常不要求由取得者釋放，記憶體排序仍依該 API 的規範。

OS06-E07　RM 充分界、EDF 與短期限

1. 甲組的利用率為 $U=2/5+4/8=0.9$。兩 task 的 Liu–Layland RM 充分界為 $2(\sqrt{2}-1)\approx0.8284$，故此充分測試不能判定通過或失敗。對 τ_2 的反應時間迭代：$w^{(0)}=4$，$w^{(1)}=4+\lceil4/5\rceil\times2=6$，$w^{(2)}=4+\lceil6/5\rceil\times2=8$，$w^{(3)}=4+\lceil8/5\rceil\times2=8$。固定點為 8，恰好等於其期限；$\tau_1$ 的反應時間為 2。
2. RM 在 [0,10) 的時間線：$J_{1,0}$ [0,2) → $J_{2,0}$ [2,5) → $J_{1,1}$ [5,7) → $J_{2,0}$ [7,8) → $J_{2,1}$ [8,10)。$J_{2,0}$ 在 [2,5) 做 3 單位後剩 1，於 t＝8 恰好完成、期限亦為 8；$J_{1,0}$、$J_{1,1}$ 分別在 2、7 完成，期限為 5、10。這段時間線核對前述固定點，不能只靠這個有限前綴聲稱無限序列均準時。
3. EDF 在每個完成或抵達事件後，選擇絕對期限最早的 ready job；括號記為（剩餘需求，絕對期限）：

時刻	事件後 ready jobs	執行至下一事件
0	$J_{1,0}$ (2,5)、$J_{2,0}$ (4,8)	$J_{1,0}$：0–2
2	$J_{2,0}$ (4,8)	$J_{2,0}$：2–5
5	$J_{2,0}$ (1,8)、$J_{1,1}$ (2,10)	$J_{2,0}$：5–6，繼續執行
6	$J_{1,1}$ (2,10)	$J_{1,1}$：6–8
8	$J_{2,1}$ (4,16)	$J_{2,1}$：8–10
10	$J_{2,1}$ (2,16)、$J_{1,2}$ (2,15)	$J_{1,2}$：10–12，搶占
12	$J_{2,1}$ (2,16)	$J_{2,1}$：12–14
14	無	idle：14–15
15	$J_{1,3}$ (2,20)	$J_{1,3}$：15–16，仍剩 1

期限不晚於 16 的 $J_{1,0}$、$J_{1,1}$、$J_{1,2}$ 分別於 2、8、12 完成，期限為 5、10、15；$J_{2,0}$、$J_{2,1}$ 分別於 6、14 完成，期限為 8、16。t＝8 時 $J_{1,1}$ 完成並釋放 $J_{2,1}$；t＝16 新 job 的到達不改變舊 $J_{2,1}$ 已完成的事實。甲組符合題設理想、可搶占的單處理器週期模型，且 D＝T、U＝0.9≤1，因此 EDF 的該模型利用率充要條件可保證後續期限；有限的 [0,16) 本身不是一般證明。 4. 乙組 U＝1，超過 0.8284。其 RM 一個共同週期為 $J_{1,0}$ [0,1) → $J_{2,0}$ [1,2) → $J_{1,1}$ [2,3) → $J_{2,0}$ [3,4)。三次完成時刻為 1、3、4，對應期限 2、4、4。t＝4 無未完成工作，且兩 task 同步釋放，狀態重現；因此此表可每 4 單位重複。充分界被超過不代表 RM 必然失敗。 5. 丙組 U＝2/10＋2/10＝0.4，但兩個 t＝0 釋放的 job 都必須在 t＝3 前各做 2 單位，共需 4 單位 CPU 時間；長度 3 的區間只有 3 單位，任何政策都至少使一個逾期。甲組 D＝T，符合上述利用率定理前提；丙組 D<T，不可僅用 U≤1 判定。若甲組加入鎖阻塞，必須另行分析阻塞上界與資源協定，原本的無阻塞理想 EDF 結論不再直接適用。

OS06-E08　到達不一定引起搶占

SRTF 時間線：P1 [0,2) → P2 [2,5) → P3 [5,6) → P1 [6,11)。t＝2 時 P1 剩 5，P2 需求 3，因此切換；t＝4 時 P2 和新到的 P3 都剩 1，依題設保留 current，不切換。

工作	完成	周轉＝完成−到達	等待＝周轉−需求	首次 CPU 回應
P1	11	11	4	0
P2	5	3	0	0
P3	6	2	1	1

SRTF 平均等待 (4+0+1)/3=5/3。非搶占 SJF 的時間線為 P1 [0,7) → P3 [7,8) → P2 [8,11)：P1 開始時沒有其他 ready 工作，之後不因新到者中斷。

工作	完成	周轉	等待	首次 CPU 回應
P1	7	7	0	0
P2	11	9	6	6
P3	8	4	3	3

非搶占 SJF 平均等待 `(0+6+3)/3=3`。本例 SRTF 的平均等待較低，但 P1 較晚完成；平均與個別結果需要分開判讀。

OS06-E09　MLFQ 被打斷的時間額度

A 在 t＝2 耗盡 Q0 的額度且仍未完成，因此降到 Q1；B 先在 Q0 執行至完成。C 在 t＝5 到達 Q0，搶占正在 Q1 執行的 A。

區間	工作／層級	該段後剩餘需求	該層額度與結果
0–2	A／Q0	5	Q0 額度耗盡，降 Q1
2–4	B／Q0	0	完成
4–5	A／Q1	4	原額度 4，已用 1，剩 3；被 C 搶占
5–6	C／Q0	0	完成
6–9	A／Q1	1	使用保留的 3，額度耗盡而降 Q2
9–10	A／Q2	0	完成

工作	完成	等待＝完成−到達−需求	首次 CPU 回應
A	10	3	0
B	4	1	1
C	6	0	0

若在 t＝6 將 A 的 Q1 額度改為 4，它會在 Q1 連續執行 [6,10) 並完成，不會於 t＝9 降至 Q2。這組工作碰巧仍在 t＝10 結束，但層級與剩餘額度不同；若另外有 Q1 工作，選取結果也可能改變，因此不能把兩種做法當成同一政策。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 6. Linux 3.2 scheduler source; Linux 3.2 scheduler features.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study